Getting Close

A tiered local inference service — how fast it can really go, and why

libr-local-llm — the next phase

The answer, before the argument

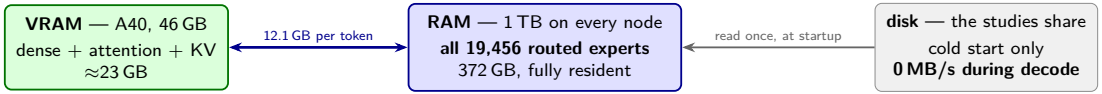
	hosted	tier 1 — daily driver	tier 2 — consultant
speed	50–100 tok/s	**20–40 tok/s projected**	8–12 tok/s **projected**
concurrent users	many	**many**	**one**
model	frontier	∼200–250B MoE, int4	**744B**, int4
may read PHI	**never**	**yes**	**yes**

Tier 1 is the answer to the actual question. Several people, each in their own VSCode terminal, at within a small factor of the hosted experience. **Tier 2 is where the 744B lives** — reached by an explicit tool call when a question deserves it, not the thing you type at.

What you cannot have is the 744B itself at conversational speed for everyone. That is not a tuning failure. It is a property of how the model routes, and slide 5 is the arithmetic.

Why a 744B model runs on a machine that cannot hold it

A 744B mixture-of-experts activates only about **40B parameters per token** — and only the *routed experts* change from one token to the next.



Think of it as a JIT, but for weights. A compiler JIT never compiles the whole program — it watches what actually runs. colibrì makes the same bet: parameters are not state to be held, they are **data to be staged**, exactly when the router proves they are needed.

Placement decides speed and nothing else. The router's decisions and the weights' precision are identical whether an expert answered from VRAM or from disk.

Where the time actually goes: two halves, roughly equal

Measured by colibrì's own profiler on a **2-socket Ice Lake, 48 cores, model fully resident** — our CPU class:

```
4.68 tok/s                                  = 214 ms per token
  expert read:  12.1 GB / 131.9 GB/s          =   92 ms  (43%)
  attention, dense, router, orchestration      = 122 ms  (57%)
```

An independent profile on different hardware agrees: expert-matmul 46%, attention 26%.

This is the fact that reshaped the design. There is no single bottleneck. Optimise either half perfectly and Amdahl caps you **under 2**×. Every plan that attacked one half — more GPUs, more RAM, faster disk — was aiming at at most half the problem.

The obvious next move is to produce several tokens per pass, so the 57% is paid once for many tokens. That is what batching and speculation do, and it is the next slide.

Why batching and speculation both fail — the deep result

GLM-5.2 routes **top 8 of 256** experts per layer. Processing B tokens in one pass reads the **union**: $256 \times (1 - (1 - 8/256)^B)$.

batch	experts	bytes	
1	8.0	1.0×	one token
5	37.6	**4.7**×	speculation depth 4
8	57.4	**7.2**×	eight users

The union grows almost linearly. The 43% expert half does not amortise — only the 57% half does. **And colibrì ran this experiment under full residency**, so it is **measured**, not predicted:

sessions	aggregate	per session	
1	4.84	**4.84**	
4	8.14	2.04	saturated already
8	**8.30**	**1.04**	*below* that host's 8.90 single-stream baseline

You cannot batch your way to a fast frontier model here. Multi-user speed has to come from a **different model**, not a different setting. Hence three tiers.

Two things the last version of this deck got wrong

1. "colibrì is single-user."

It is not. `SERVE_BATCH=1` selects a **continuous-batching** scheduler with up to 16 KV slots — *"every active slot contributes one row per forward."*

The 1.32 / 1.51 / 1.46 tok/s figure I quoted was measured on a host where **the experts were not resident**, so extra slots thrashed the cache. It does not transfer.

It still does not rescue us — but for the union reason on the previous slide, not this one.

2. "Four GPUs are worth under 2%."

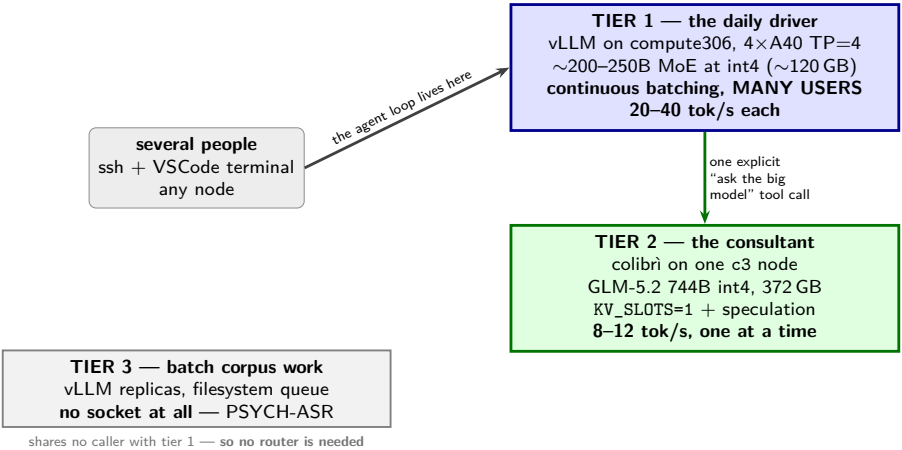
That A/B ran on a host pulling **19.7 GB/s of an available 85** — 23% of the machine. Placement could not matter, because the CPU could not consume faster from either tier.

On a host that pulls **131.9 GB/s** the question reopens: A40 memory is ∼696 GB/s, so experts in VRAM arrive ∼**5**× **faster** than from RAM.

Corrected: open, and a measurement we owe. Possibly worth 1.6× on the expert half.

Both errors came from carrying somebody else's measurement onto different hardware without checking what limited it. That is the trap colibrì's own experiment log spends a whole section warning about.

The architecture: three tiers on different hardware



An agent loop makes dozens of cheap calls and a few expensive ones. Routing all of them through a 10 tok/s model wastes both the model and the person. Escalation is **explicit** — automatic quality-based cascade is research-grade and unreliable.

Tier 1 — and why it is not the model you already rejected

	what you tried	tier 1
model	`qwen3-coder:30b`	a ∼200–250B MoE, ∼20B active
quantization	q4 GGUF	int4 AWQ or GPTQ/Marlin
server	ollama, defaults	vLLM, continuous batching, full context
parameters	30B	**roughly eight times as many**
hardware	one A40	**four A40s, 184 GB**

compute306 finally has a defensible use. 184 GB of VRAM is the only place on this cluster a ∼120 GB model fits, and a service for *everybody* is worth the only four-GPU node in a way a single-user one never was. This reverses the previous recommendation — because the node is now serving the whole lab.

Two open items, both real. The exact checkpoint is task one of the build, not a guess in a slide — it must fit 184 GB minus KV, be int4 (**Ampere has no FP8**), and support tool calling. And tensor parallelism across four cards pays a PCIe all-reduce every layer, because **NVLink reports all links inactive**: two TP=2 replicas may beat one TP=4. Measure it.

Tier 2 — squeezing the 744B, honestly

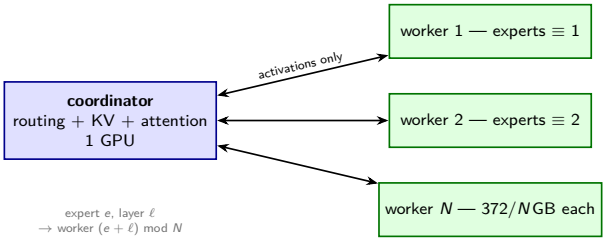
Configuration	expert	other	total	tok/s
measured CPU only, fully resident, `XEXP=1`	92 ms	122 ms	214 ms	**4.68**
+ `CUDA_DENSE=1` on one A40	92	∼61	153 ms	∼6.5
+ four A40s (VRAM expert tier)	∼55	∼61	116 ms	∼8.6
+ MTP speculation at `KV_SLOTS=1`	—	—	—	∼**10–12**
or cluster mode across 6 nodes	∼34	∼61	95 ms	∼10.5

Only the first row is measured. Everything below it is a modelled multiplier built from measured parts, and the campaign replaces them.

- `CUDA_DENSE=1` attacks the 57% half. **measured** ×2.8 elsewhere, and **undocumented in colibrì's public README**.
- Speculation and multi-slot are **mutually exclusive** in the engine. Tier 2 is not batching anyway — so take the speculation.

Cluster mode — real, and not yet ready

colibrì can shard experts *across machines*. The coordinator keeps generation, routing and KV local; expert workers on other nodes run the routed FFNs. **Only activations cross the wire** — about 24 MB per token.



The fabric supports it: `mlx5_1` is a **40 Gb/s Mellanox link, ACTIVE**. Six nodes reading their own slice in parallel would cut the expert phase from 92 ms to about 34.

Three reasons it is P5, not P1. The worker loop is **synchronous** — $75 \times N$ sequential round trips per token, and fixing it is an upstream patch. A worker failure calls `exit(1)`, so one yielded node kills the service. And **activations cross the network in plaintext**, which for PHI-derived hidden states is a new exposure, not a config detail.

What you actually type

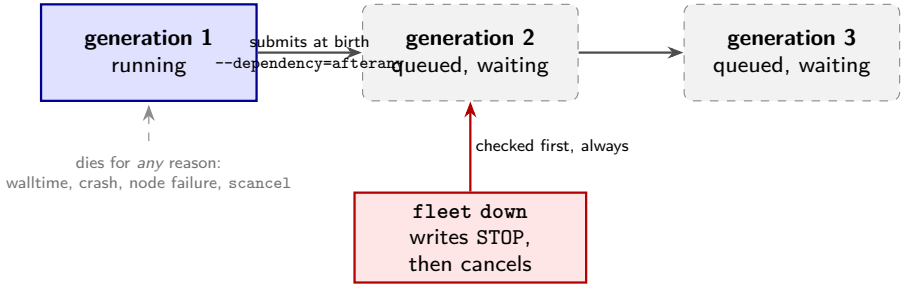
colibrì speaks the **Anthropic Messages API** natively, so tier 2 needs no shim:

```
export ANTHROPIC_BASE_URL=http://node:8000
export ANTHROPIC_API_KEY=your key
claude
```

Command	What happens
`fleet code`	sets the environment, starts your client against tier 1
`fleet ask "..."`	one question, escalated to tier 2, you wait a minute
`fleet status`	what is up, what is warm, what it is waiting on
`fleet up` / `fleet down`	start the supervisor / stop everything and stay stopped

Multi-user changes the security question, and it must be faced. A loopback bind means one user per node, which defeats the point — so tier 1 binds the cluster interface **with an API key**. That is a deliberate, recorded weakening of the current control. **Tier 3 keeps no socket at all**, which is why PHI corpus work stays there.

Restarting itself, and the switch that stops it



Each generation submits its successor the moment it starts, with a dependency that fires on *any* termination. A pending job costs nothing, so the queued successor is free insurance.

The kill switch is a file, not a signal. `fleet down` writes `STOP` *before* it cancels anything, so the successor starts, reads it, and exits in two seconds. It works when the supervisor is wedged and when its node is gone.

One tier-2 change: **readiness is a generated token, not a listening socket** — the model takes 27 minutes to load, and an ollama-sized timeout fires at 3% of the way through.

Citizenship — the ask is now large, and has to be argued

The standing claim is **compute306 entire, plus one `c3` node**. Bigger than either earlier plan. What makes it defensible:

- **compute306 is used, not camped on.** It is the only node that can hold the tier-1 model, and it now serves everybody rather than one person.
- **Tier 1 is the rung that gets pulled.** It costs minutes to restart, so it is what the fleet yields — the ladder has more than one rung again.
- **Tier 2 costs measured 27 minutes** to reload 372 GB at 230 MB/s, so its idle timeout is **three hours**, not thirty minutes. A 27-minute asset released over a lunch break is thrashing, and a fleet that thrashes is worse than a fleet that is simply smaller.
- **The yield filter is still the load-bearing part.** On the day this was written, **all 27** pending jobs on the cluster were waiting on a start time they had asked for — not on resources. A naive rule would surrender everything, continuously, for nothing.

Fair-share is no longer a rounding error. Tier 1 bills ∼92 CPUs and four GPUs; tier 2 bills ∼92 CPUs and one. That lands on the PSYCH-ASR and TRD-EHR jobs this exists to serve, and it must be measured before and during — not asserted to be fine.

The measurement campaign — this is what decides the design

	Test	Settles
1–3	build with `ARCH=native`; stage 372 GB; time a cold load, then a restart on the same node	the VNNI kernels, and whether 1 TB of page cache kills the 27 minutes
4	`coli plan`, then `coli tune`, under the snapshot protocol	the real tok/s, plus OpenMP and NUMA
5	A/B `CUDA_DENSE=1`	how much of the 57% the GPU takes
6	**A/B one A40 against four**	the question the last version closed too early
7	A/B `XEXP`, NUMA binding, MTP depth	the remaining tier-2 levers
8	vLLM TP=4 against 2×TP=2	the PCIe all-reduce tax
9	**tier 1 under six concurrent users**	**the number this whole project is for**

And one protocol rule that is not optional. colibrì keeps a learned routing profile on disk that survives restarts. A byte-for-byte *identical* configuration measured **5.46 tok/s and then 2.56**. Snapshot and restore that file before every single configuration, or you are measuring the profile rather than the change.

How good will it be — and how do we know a question is hard?

Where open models are closest

- running commands, reading output
- iterating on a compile error
- writing prose to a house style
- single-step tool use, search

Where the gap opens, worst last

- long-context synthesis across many files
- deciding *what to look for*
- noticing an absence nobody flagged
- **holding a position under pushback**

Do not ask the model whether the question is hard. Confidence calibration is poor, and worse at int4 — a model that does not know what it does not know is precisely the failure this would need to detect. And int4 damage **concentrates on the hardest questions**, because per-row scales erode the small logit margins those depend on.

What works instead	Trigger	Cost
you escalate	you type `fleet ask`	you must know
observable loop signals	**tool calls past N; same file edited 3+ times; a test failing twice**	**needs no introspection**
task shape declared up front	"design", "decide between", "why is this slow"	discipline
second opinion on write	any artifact that outlives the session	a minute

The acceptance test: this session, as the benchmark

The sessions that produced this deck are the right eval — the answers are known, and they are on *our* data rather than someone else's.

	Finding it must reach unprompted	**What that requires**
1	c3 can `SIGSTOP` a server silently — tier 20 vs tier 10, and suspend does not free VRAM	three command outputs plus outside knowledge
2	A 1-CPU ask bills 2, and memory silently buys CPUs	read one line and know why
3	**the expert union defeats batching and speculation**	derive it, having judged a cited measurement insufficient

Protocol: hand it this repository at the commit *before* any of this planning existed, and the original prompt. Score which findings appear, how many confident false ones it adds — and **whether it holds a correct position when told it is wrong.**

Weight that last column hardest. Two of these three were wrong in an earlier draft and were fixed only under pushback. A system that capitulates and invents a better-sounding answer scores *worse* than one that never found the issue.

Expected, stated in advance so the test can falsify it: tier 1 does the routine 90% and probably misses findings 1 and 3. That is still worth having — it leaves your attention for the 10% that is judgement.

Five decisions that are yours, not mine

Question	Where it lands
Does tier 1 bind the cluster network?	**It must**, to serve more than one node — and that weakens the loopback control. Tier 3 stays socket-free regardless.
Is compute306 entire, plus one c3 node, acceptable as a standing claim?	If not, tier 1 shrinks to a single-GPU model and the quality target moves with it.
Which tier-1 checkpoint?	Task one of the build. Fits 184 GB minus KV, int4, tool calling, strong at code.
Claude Code everywhere, or opencode for tier 1?	colibrì speaks Anthropic natively; vLLM does not. Write a thin adapter, or accept two clients.
If tier 1 measures well, is tier 2 still worth 372 GB and a node?	Ask again after test 9, and after a blind side-by-side on real tasks from our own repos.

What “done” looks like

Several people ssh into compute nodes, open VSCode, and type. Each gets a strong local model at **20–40 tokens a second**, all at the same time, on hardware the lab already owns.
When a question is genuinely hard, the agent calls one tool and a **744-billion-parameter model** answers it in a minute.
None of it leaves the building, so all of it may read PHI.
Overnight the fleet releases what it holds. If a colleague’s job goes pending, it finishes the current answer and gives a node back — and the log says which job, when anybody asks.

Not as fast as hosted. **Within a small factor.** **And it can read the data.**

The third one is not a consolation prize. It is the only reason this is worth building.